

Universidade de Brasília
Instituto de Ciências Exatas
Departamento de Ciência da Computação

HySAIL: título a definir

Autor HySAIL

Dissertação apresentada como requisito parcial para qualificação do
Mestrado Profissional em Computação Aplicada

Orientador
Prof. Dr. Orientador a definir

Brasília
2026



Universidade de Brasília
Instituto de Ciências Exatas
Departamento de Ciência da Computação

HySAIL: título a definir

Autor HySAIL

Dissertação apresentada como requisito parcial para qualificação do
Mestrado Profissional em Computação Aplicada

Prof. Dr. Orientador a definir (Orientador)
CIC/UnB

Prof. Dr. Avaliador a definir
Instituição a definir

Prof. Dr. Avaliador a definir
Instituição a definir

Prof.a Dr.a Coordenação a definir
Coordenadora do Programa de Pós-graduação em Computação Aplicada

Brasília, 14 de outubro de 2026

Resumo

Palavras-chave: palavras-chave a definir

Abstract

Keywords: keywords to be defined

Contents

1 Apresentação **1**

1.1 Objetivos 4

1.2 Contribuições Esperadas 4

1.3 Organização do Trabalho 4

2 Fundamentação Teórica e Trabalhos Relacionados **5**

2.1 Conceitos Fundamentais 5

2.2 Corpos finitos e corpo de Galois 5

2.2.1 Exemplo de representação polinomial 5

2.3 Divisão polinomial em GF(2) 6

2.3.1 Exemplo de divisão passo a passo 6

2.3.2 HySAIL e Codificação de Blocos 7

2.3.3 LT Codes 7

2.3.4 RaptorQ Codes 8

2.3.5 Online Codes 10

3 Proposta **12**

4 Metodologia de Pesquisa **13**

4.1 Provas de Integridade e Disponibilidade 13

4.2 Avaliação de RaptorQ Codes 14

5 Implementação **15**

6 Resultados **16**

7 Conclusão **17**

Referências **18**

Acronyms

EB Exabyte.

PB Petabyte.

ZB Zettabyte.

Chapter 1

Apresentação

Desde a proposta do HySAIL, em 2013, o cenário tecnológico passou por transformações significativas. A expansão da computação em nuvem, dos dispositivos móveis, da Internet das Coisas e dos serviços digitais alterou a forma como os dados são produzidos, armazenados, processados e compartilhados. Nesse contexto, abordagens concebidas naquele período devem ser analisadas à luz das novas demandas tecnológicas, dos diferentes ambientes de processamento e das atuais expectativas de segurança e privacidade.

O crescimento da produção de dados está ligado à digitalização de atividades cotidianas e científicas. Interações em redes sociais, transações comerciais, sensores conectados, serviços de streaming e experimentos científicos geram registros em diferentes formatos e de forma contínua. Para dimensionar essa produção, é importante distinguir entre dados publicados ou coletados, tráfego transmitido e dados acumulados em armazenamento: são métricas relacionadas, mas não equivalentes.

O repositório *Amazon Simple Storage Service* (S3) registrava mais de 100 trilhões de objetos acumulados até 2021, totalizando cerca de *500 EB* armazenados. O tráfego de vídeos consumidos na Netflix gerou aproximadamente *51,1 EB* em 2021[1], enquanto as trocas globais de e-mails e spam movimentaram cerca de *5,7 EB* no mesmo período. O volume anual de conteúdos criados no YouTube atinge cerca de *263 PB*, ao passo que o compartilhamento de imagens no Facebook e no Instagram representa *252 PB* e *68 PB*, respectivamente [1].

Nos experimentos de física de altas energias do CERN, os sistemas eletrônicos de leitura do *Large Hadron Collider* (LHC) geraram cerca de *40 ZB* de dados brutos de colisão em 2018[1]. Como é inviável armazenar todo esse volume, sistemas de seleção (*triggers*) filtram o fluxo para gravar cerca de *160 PB* por ano (∼240 PB, incluindo simulações de Monte Carlo)[1]. Com o projeto *High-Luminosity LHC* (HL-LHC), prevê-se o registro de *800 PB* de novos dados por ano a partir de 2026[1]. Além disso, a rede de distribuição *Worldwide LHC Computing Grid* (WLCG) atingiu uma vazão de 60 GB/s,

movimentando cerca de *1,9 EB* por ano[1].

O crescimento desses fluxos também favorece a expansão de sistemas de inteligência artificial (IA), que dependem de dados para treinamento, validação e operação. Segundo o *AI Index Report 2025*, a proporção de organizações que declararam utilizar IA em pelo menos uma função empresarial passou de 55% em 2023 para 78% em 2024. A adoção dessas tecnologias amplia as possibilidades de análise e automação, mas torna indispensáveis mecanismos de governança, controle de acesso e proteção de dados pessoais, a fim de reduzir riscos à privacidade, à segurança e à confiabilidade dos sistemas e de seus resultados.

O volume de dados utilizado para treinar modelos de IA de ponta aumentou cerca de *30.000 vezes desde 2008*[2]. Desde 2014, o volume de processamento computacional aumentou 350.000 vezes. Como exemplo, o modelo GPT-4 exigiu cerca de 4,9 trilhões de pontos de dados para seu treinamento. Projeta-se que, até 2035, o tráfego de inferência de IA passe a responder por 25% de todo o tráfego de rede mundial.

Nesse cenário de grande produção e consumo de dados, soluções estabelecidas continuam a fundamentar tecnologias atuais. Entre elas, destaca-se o armazenamento em nuvem, um modelo de computação que permite guardar, gerenciar e acessar dados remotamente por meio da Internet, em vez de mantê-los exclusivamente em dispositivos físicos ou servidores locais [2]. Entre suas vantagens, destacam-se:

- *Escalabilidade e Elasticidade:* permite expandir ou reduzir a capacidade de armazenamento de forma dinâmica à medida que o volume de dados cresce, sem a necessidade de gerenciar diretamente o hardware subjacente [2].
- *Redução de Custos:* minimiza despesas com infraestrutura local, manutenção física e aquisição de servidores próprios [3].
- *Capacidade de Acesso Global:* os dados armazenados em centros de dados distribuídos podem ser acessados de qualquer localidade com suporte a conexões de alta velocidade[3].
- *Suporte a Múltiplas Arquiteturas de Dados:* abrange desde o armazenamento de objetos (como Amazon S3) até sistemas de arquivos distribuídos (como o Hadoop Distributed File System – HDFS) e Data Lakes, capazes de armazenar grandes volumes de dados brutos e estruturados [2].

Diante desse cenário de grande volume de dados, persistem diversos problemas relacionados à sua segurança, tais como:

- *Falhas de Anonimização e Ataques de Reidentificação:* dados aparentemente anonimizados por remoção de identificadores diretos permanecem vulneráveis a ataques

de vinculação (*linkage attacks*)[4]. Estudos demonstram que apenas quatro pontos espaço-temporais em bases de mobilidade ou histórico de compras são suficientes para reidentificar entre 90% e 95% dos indivíduos quando cruzados com outras fontes [4].

- *Ameaça do Aprendizado de Máquina e Mineração de Dados:* o avanço dos algoritmos de *data mining* e *machine learning* permite inferir e extrair conhecimentos sensíveis não autorizados a partir da correlação de bases de dados aparentemente inofensivas.
- *Limitações das Técnicas Tradicionais de Agrupamento:* modelos clássicos de privacidade em bancos de dados, como k-*anonymity*, são suscetíveis a ataques de homogeneidade e de conhecimento prévio (*background knowledge attacks*)[4]. Extensões como l-*diversity* e t-*closeness* foram criadas para mitigar essas falhas, aumentando a variação de atributos sensíveis e limitando o ganho de informação por invasores[4].
- *Gargalos Criptográficos e Ataques de Informação Lateral:* a criptografia convencional enfrenta barreiras de escalabilidade e alto custo computacional em ambientes de Big Data e dispositivos móveis[4]. Soluções como *Criptografia Baseada em Atributos (ABE)* e *criptografia pesquisável* buscam integrar o controle de acesso e a busca em dados cifrados[4]. Além disso, a criptografia do conteúdo não impede ataques de análise de tráfego (*side information attacks*), que identificam padrões e "pegadas" (*fingerprints*) de navegação[4].
- *Vulnerabilidades em Fluxo (*Streaming*) e Armazenamento:* dados em transmissão contínua (*streaming*) estão sujeitos a vazamentos na origem, antes da cifragem; à interceptação e à adulteração durante a transferência pela rede; e à perda de integridade ou confiabilidade nos repositórios de armazenamento[5].
- *Ameaças Cibernéticas a Infraestruturas Críticas:* redes de sensores IoT, ambientes industriais e *Data Lakes* tornaram-se alvos diretos de programas maliciosos, intrusões, *phishing*, e-mails enganosos e ataques de negação de serviço (DDoS)[6].
- *Dilemas Éticos, Legais e de Perfilamento:* a coleta massiva levanta questionamentos sobre o consentimento implícito e o explícito [7]. O uso de análise preditiva para perfilamento (*profiling*) traz riscos de discriminação e decisões injustas[20], exigindo conformidade com regulamentações, como a GDPR, e salvaguardas como o "direito ao esquecimento"[4].

Portanto, embora o cenário tecnológico tenha evoluído, permanece necessária a adoção de ferramentas que restrinjam o acesso indevido a dados que precisam ser protegidos.

1.1 Objetivos

A princípio, a fundamentação de HySAIL é dependente de Online Codes para a geração de blocos que são distribuídos. Considerando novas otimizações aplicadas em RaptorQ Codes [8], o presente trabalho visa explorar a utilização do mesmo para o processo de codificação e decodificação de blocos.

Após a implementação e a avaliação dessas mudanças, serão realizadas comparações com soluções adotadas anteriormente, a fim de identificar os ganhos proporcionados pelo novo modelo.

1.2 Contribuições Esperadas

Ao final deste trabalho, espera-se que o modelo proposto para codificação e decodificação de blocos utilizando RaptorQ Codes apresente melhorias em termos de eficiência, segurança e robustez em comparação com as soluções baseadas em Online Codes e que exista prova matemática que admita seu uso.

Além disso, espera-se que as contribuições deste estudo forneçam subsídios para a adoção de técnicas avançadas de codificação em cenários de Big Data e ambientes distribuídos.

1.3 Organização do Trabalho

O presente trabalho está organizado da seguinte forma:

- No Capítulo 2, são apresentados os conceitos fundamentais e o estado da arte relacionado à codificação de blocos e RaptorQ Codes.
- O Capítulo 3 descreve a metodologia adotada para a implementação do modelo proposto.
- No Capítulo 4, são apresentados os resultados obtidos e a análise comparativa com soluções baseadas em Online Codes.
- Finalmente, o Capítulo 5 apresenta as conclusões e as perspectivas para trabalhos futuros.

Chapter 2

Fundamentação Teórica e Trabalhos Relacionados

2.1 Conceitos Fundamentais

Antes de detalhar o modelo proposto, é importante compreender alguns conceitos matemáticos relacionados à codificação de blocos. Por fim, para que possamos compreender a nova abordagem adotada, é necessário revisar o funcionamento de HySAIL, LT Codes e RaptorQ Codes.

2.2 Corpos finitos e corpo de Galois

O corpo de Galois é uma estrutura algébrica que define operações e suas propriedades sobre um conjunto finito de elementos. No contexto do HySail, destacam-se o corpo $GF(2)$ e suas extensões $GF(2^n)$, que permitem representar e manipular dados binários por meio de operações matematicamente bem definidas.

Em $GF(2)$, os únicos elementos são 0 e 1, e a soma segue a mesma lógica da operação XOR, ou seja, o resultado é 0 quando os operandos são iguais e 1 quando são diferentes. Assim, $1 + 1 = 0$ e $1 + 0 = 1$. As extensões $GF(2^n)$, por sua vez, permitem operar sob elementos representados por combinações de n coeficientes binários.

2.2.1 Exemplo de representação polinomial

Considere o vetor binário 1011. Adotando a convenção usada pelo projeto, em que a primeira posição do vetor representa o termo constante, o bloco é lido em três passos:

1. o primeiro coeficiente vale 1 e representa 1;

2. o segundo coeficiente vale 0 e representa $0x$;

3. os coeficientes seguintes valem 1 e 1, representando x^2 e x^3.

Assim, o bloco passa a ser escrito como $1 + x^2 + x^3$.

2.3 Divisão polinomial em $GF(2)$

A divisão polinomial em $GF(2)$ segue a mesma lógica estrutural da divisão entre inteiros, mas com uma diferença importante: como os coeficientes pertencem a um corpo binário, subtração e soma são equivalentes a XOR. Portanto, ao somar dois termos iguais, eles se anulam: $x^k + x^k = 0$.

Dado um dividendo $m(x)$ e um divisor $p(x)$, procura-se obter um quociente $q(x)$ e um resto $r(x)$ tais que

$$m(x) = q(x) \cdot p(x) + r(x),$$

com $\deg(r) < \deg(p)$.

O procedimento é repetido enquanto o resto parcial tiver grau maior ou igual ao grau do divisor. Em cada iteração, divide-se o termo de maior grau do resto parcial pelo termo de maior grau do divisor. O resultado é o próximo termo do quociente. Em seguida, esse termo multiplica o divisor, e aplica-se XOR entre o produto e o resto parcial. Essa operação elimina o termo de maior grau, reduzindo o grau do novo resto.

2.3.1 Exemplo de divisão passo a passo

Seja $m(x) = x^3 + x + 1$ o dividendo e $p(x) = x^2 + 1$ o divisor. Inicialmente, o resto parcial é o próprio dividendo:

$$r_0(x) = x^3 + x + 1.$$

1. O termo de maior grau de $r_0(x)$ é x^3 e o de $p(x)$ é x^2. Logo, o próximo termo do quociente é $x^3/x^2 = x$.

2. Multiplica-se esse termo pelo divisor: $x \cdot p(x) = x(x^2 + 1) = x^3 + x$.

3. Aplica-se XOR entre o resto parcial e o produto:

$$r_1(x) = (x^3 + x + 1) + (x^3 + x) = 1.$$

Os termos x^3 e x se anulam porque cada um aparece duas vezes.

4. Como $\deg(1) = 0$ é menor que $\deg(p) = 2$, a divisão termina. O quociente é $q(x) = x$ e o resto é $r(x) = 1$.

Substituindo o quociente e o resto obtidos na identidade da divisão, verifica-se que

$$x^3 + x + 1 = x(x^2 + 1) + 1.$$

Pela convenção em que o primeiro bit representa o coeficiente do termo constante, o quociente $q(x) = x$ tem representação binária 01, e o resto $r(x) = 1$ tem representação binária 1. Caso o resto seja armazenado com dois bits, que é o número de coeficientes de grau menor que o divisor de grau 2, sua representação é 10.

2.3.2 HySAIL e Codificação de Blocos

Empty

2.3.3 LT Codes

Os **LT Codes** (*Luby Transform Codes*), propostos por Luby [9], são a primeira realização prática dos códigos *fountain* digitais. Por não possuírem taxa fixa, permitem gerar, a partir de k símbolos de origem, uma quantidade potencialmente ilimitada de símbolos codificados, dos quais qualquer subconjunto com pouco mais de k elementos basta para reconstruir os dados, sem necessidade de retransmissão seletiva.

Seja $\mathbf{d} = (d_1, \ldots, d_k)$ o vetor de símbolos de origem, cada um correspondente a um bloco de bytes do arquivo. Para gerar o símbolo codificado c_i, sorteia-se um grau δ_i segundo uma distribuição de grau e selecionam-se uniformemente δ_i símbolos de origem distintos, cujos índices formam o conjunto S_i. O símbolo é então dado por

$$c_i = \bigoplus_{j \in S_i} d_j, \tag{2.1}$$

em que $\bigoplus$ denota a soma em $GF(2)$, isto é, o XOR bit a bit. Para que o receptor conheça S_i, transmite-se junto a c_i a semente do gerador pseudoaleatório utilizado, permitindo reproduzir as escolhas do codificador.

A distribuição de grau determina o desempenho do código: símbolos de grau baixo são necessários para iniciar e manter a decodificação, enquanto símbolos de grau elevado garantem que todos os símbolos de origem sejam cobertos [9]. Seu projeto busca equilibrar esses requisitos, minimizando o *overhead* de recepção, isto é, o número de símbolos adicionais necessários à recuperação.

A decodificação utiliza o algoritmo iterativo de *belief propagation*, ou *peeling*. A cada iteração, um símbolo codificado de grau unitário fornece diretamente um símbolo de

origem d_j, que é somado aos demais símbolos codificados que o contêm, reduzindo seus graus e podendo originar novos símbolos de grau unitário. O processo termina quando os k símbolos são recuperados ou, na ausência de símbolos de grau unitário, é suspenso até a chegada de novos símbolos codificados.

Exemplo de codificação e decodificação

Considere três símbolos de origem de quatro bits:

$$d_1 = 1010, \quad d_2 = 0110, \quad d_3 = 1101. \tag{2.2}$$

Com os conjuntos sorteados $S_1 = \{2\}$, $S_2 = \{1, 2\}$ e $S_3 = \{1, 2, 3\}$, a Equação (2.1) fornece:

$$c_1 = d_2 = 0110, \quad c_2 = d_1 \oplus d_2 = 1100, \quad c_3 = d_1 \oplus d_2 \oplus d_3 = 0001. \tag{2.3}$$

Na decodificação, c_1 fornece diretamente d_2. Removendo d_2 dos demais símbolos, obtêm-se $c_2 \oplus d_2 = d_1 = 1010$ e $c_3 \oplus d_2 = d_1 \oplus d_3 = 0111$. Por fim, $0111 \oplus d_1 = 1101 = d_3$, completando a recuperação.

2.3.4 RaptorQ Codes

RaptorQ Codes pertencem à família dos códigos *fountain* sem taxa fixa. Isso significa que, a partir de um conjunto de símbolos de origem, o codificador pode produzir uma quantidade potencialmente ilimitada de símbolos codificados. O receptor não precisa receber símbolos específicos nem todos os símbolos transmitidos: basta obter uma quantidade suficiente de símbolos distintos para reconstruir os dados originais. Essa característica torna o código adequado a cenários com perdas de pacotes, como a transmissão em redes não confiáveis e o armazenamento distribuído [10, 8].

O processo de codificação é composto por duas etapas. Primeiro, os k símbolos de origem são submetidos a uma pré-codificação, que acrescenta símbolos de reparo e estabelece relações entre os símbolos do bloco. Em seguida, uma codificação LT gera cada símbolo de saída pela combinação de um subconjunto dos símbolos intermediários. Assim como nos LT Codes, essas combinações usam operações XOR em $GF(2)$ quando os símbolos são binários.

De forma simplificada, seja $\mathbf{u} = (u_1, u_2, \ldots, u_k)$ o vetor de símbolos de origem. A pré-codificação produz um vetor intermediário $\mathbf{v}$, e cada símbolo codificado c_i é obtido por:

$$c_i = \bigoplus_{j \in S_i} v_j, \tag{2.4}$$

em que S_i é o conjunto de índices selecionados para o símbolo c_i. A diferença em relação ao LT Code isolado é que as relações introduzidas na pré-codificação permitem recuperar símbolos de origem mesmo quando o processo iterativo não encontra imediatamente símbolos de grau 1.

Exemplo simplificado de codificação e recuperação

Considere um bloco com três símbolos de origem de quatro bits:

$$u_1 = 1010, \quad u_2 = 0110, \quad u_3 = 1101. \tag{2.5}$$

Para fins didáticos, suponha que a pré-codificação produza o símbolo de reparo $v_4 = u_1 \oplus u_2 \oplus u_3$. Assim, os símbolos intermediários são $v_1 = u_1$, $v_2 = u_2$, $v_3 = u_3$ e:

$$v_4 = 1010 \oplus 0110 \oplus 1101 = 0001. \tag{2.6}$$

Em seguida, a etapa LT pode gerar símbolos codificados a partir de subconjuntos desses símbolos intermediários. Considere os seguintes símbolos transmitidos:

$$c_1 = v_1 = 1010, \quad c_2 = v_2 \oplus v_3 = 1011, \quad c_3 = v_2 \oplus v_4 = 0111. \tag{2.7}$$

Suponha que o símbolo de origem u_2 não seja recebido diretamente. A partir de c_1, o receptor recupera u_1. Além disso, como conhece a relação de pré-codificação e recebe c_2 e c_3, pode recuperar os símbolos ausentes. Primeiro, observa-se que:

$$c_2 \oplus c_3 = (v_2 \oplus v_3) \oplus (v_2 \oplus v_4) = v_3 \oplus v_4 = 1100. \tag{2.8}$$

Pela relação de pré-codificação, $v_3 \oplus v_4 = u_3 \oplus (u_1 \oplus u_2 \oplus u_3) = u_1 \oplus u_2$. Portanto, como u_1 foi recuperado de c_1, obtém-se:

$$u_2 = (c_2 \oplus c_3) \oplus u_1 = 1100 \oplus 1010 = 0110. \tag{2.9}$$

Esse exemplo ilustra o papel da pré-codificação: relações adicionais entre os símbolos permitem recuperar dados perdidos a partir dos símbolos codificados recebidos. O RaptorQ completo utiliza relações e distribuições de grau definidas pelo padrão, mas o princípio de recuperação por combinações XOR é o mesmo.

Na decodificação, o receptor reúne os símbolos codificados recebidos e suas relações de codificação. Inicialmente, são aplicadas eliminações XOR para resolver os símbolos

que podem ser obtidos diretamente. As relações restantes são então resolvidas em conjunto com as informações da pré-codificação. Desse modo, o RaptorQ combina a geração flexível de símbolos dos LT Codes com uma camada adicional de proteção, reduzindo a probabilidade de falha na recuperação quando são recebidos símbolos suficientes [10, 8].

Na prática, essa propriedade é expressa pelo *overhead* de recepção: a quantidade de símbolos adicionais que pode ser necessária além dos k símbolos de origem. Um *overhead* pequeno permite recuperar o bloco mesmo após perdas, sem exigir a retransmissão de símbolos específicos. Entretanto, a etapa adicional de pré-codificação e a resolução das relações remanescentes aumentam a complexidade de decodificação em comparação com um LT Code simples [8].

2.3.5 Online Codes

Os **Online Codes**, propostos por Maymounkov [11, 12], são códigos *fountain* que, assim como os LT Codes, geram uma quantidade potencialmente ilimitada de símbolos codificados por meio de combinações XOR. Diferentemente destes, porém, empregam uma distribuição de grau com grau máximo constante, independente de k, o que resulta em codificação e decodificação de complexidade linear. Para compensar a recuperação apenas parcial que essa distribuição proporciona, os Online Codes combinam dois níveis de codificação: um código externo e um código interno.

O código externo (*outer code*) acrescenta redundância aos k símbolos de origem $\mathbf{d} = (d_1, \ldots, d_k)$. Cada símbolo de origem é associado aleatoriamente a q dos m símbolos auxiliares $a_1, \ldots, a_m$, sendo cada símbolo auxiliar a soma dos símbolos de origem a ele associados:

$$a_l = \bigoplus_{j \in A_l} d_j, \tag{2.10}$$

em que A_l é o conjunto de índices dos símbolos de origem associados a a_l. Os símbolos de origem e auxiliares formam a mensagem composta $\mathbf{v} = (d_1, \ldots, d_k, a_1, \ldots, a_m)$.

O código interno (*inner code*) gera os símbolos codificados, denominados blocos de verificação (*check blocks*), de forma análoga à Equação (2.1), porém sobre a mensagem composta: sorteia-se um grau segundo a distribuição de grau e combinam-se, via XOR, símbolos de $\mathbf{v}$ escolhidos uniformemente.

A decodificação ocorre em duas etapas. Inicialmente, o algoritmo de *peeling* descrito para os LT Codes é aplicado aos blocos de verificação recebidos, recuperando uma fração dos símbolos da mensagem composta próxima de 1, mas não necessariamente todos. Em seguida, as relações do código externo, dadas pela Equação (2.10), são utilizadas para recuperar os símbolos de origem remanescentes. Com $\varepsilon > 0$ como parâmetro de projeto,

a recuperação completa é obtida com aproximadamente $(1 + 3\varepsilon)k$ blocos de verificação, com probabilidade de falha que decresce exponencialmente com q [11].

Exemplo de codificação e decodificação

Considere os mesmos símbolos de origem do exemplo anterior, $d_1 = 1010$, $d_2 = 0110$ e $d_3 = 1101$, e um código externo com um único símbolo auxiliar associado a todos eles:

$$a_1 = d_1 \oplus d_2 \oplus d_3 = 0001. \tag{2.11}$$

Suponha que o receptor obtenha os seguintes blocos de verificação, gerados a partir da mensagem composta $\mathbf{v} = (d_1, d_2, d_3, a_1)$:

$$c_1 = d_1 = 1010, \quad c_2 = d_3 \oplus a_1 = 1100, \quad c_3 = a_1 = 0001. \tag{2.12}$$

Pelo código interno, c_1 e c_3 fornecem diretamente d_1 e a_1, e $c_2 \oplus a_1 = 1101 = d_3$. O símbolo d_2, que não participa de nenhum bloco recebido, é então recuperado pelo código externo: $d_2 = a_1 \oplus d_1 \oplus d_3 = 0110$.

Chapter 3

Proposta

Chapter 4

Metodologia de Pesquisa

A metodologia de pesquisa adotada neste trabalho foi divida em etapas distintas que possibilitam uma análise detalhada do impacto da substituição de Online Codes por RaptorQ Codes na integridade, disponibilidade e eficiência de codificação e decodificação dos dados.

Antes de realizar a proposta de substiuição, foi realizada uma revisão da literatura sobre Online Codes e RaptorQ Codes, bem como uma análise das soluções existentes que utilizam esses códigos para garantir a integridade e disponibilidade dos dados. O objetivo é compreender o estado da arte e identificar lacunas que justificam a substituição de Online Codes por RaptorQ Codes.

As bases de dados utilizadas para a revisão da literatura incluem Scopus, Web of Science, Google Scholar e IEEE Xplore. As palavras-chave utilizadas na busca incluem "Online Codes", "RaptorQ Codes", "proof of retrievability", "erasure codes", "data integrity" e "data availability", bem como seus sinônimos e variações em português e inglês. A busca foi realizada de forma sistemática, considerando artigos publicados nos últimos 8 anos para garantir a relevância e atualidade das informações.

Após a revisão da literatura e a análise das soluções existentes, foi possível identificar lacunas e oportunidades para a substituição de Online Codes por RaptorQ Codes, justificando a proposta apresentada neste trabalho.

4.1 Provas de Integridade e Disponibilidade

O que fundamenta HySAIL são suas provas de integridade e disponibilidade dos dados, que garante que os dados armazenados permanecem corretos e acessíveis mesmo diante de falhas ou ataques maliciosos. Contudo, a prova considera um cenário onde Online Codes são utilizados, sendo necessário avaliar como a substituição por RaptorQ Codes impacta essas garantias. Essa avaliação envolverá a análise teórica das propriedades de RaptorQ

Codes em relação à integridade e disponibilidade, bem como experimentos que simulem falhas e ataques para verificar se as garantias são mantidas.

4.2 Avaliação de RaptorQ Codes

Considerando o cenário onde RaptorQ Codes pode substituir Online Codes, é necessário avaliar o impacto dessa substituição na eficiência de codificação e decodificação, bem como na robustez frente a perdas de pacotes. Serão realizados experimentos que medem o overhead de recepção, o tempo de decodificação e a taxa de recuperação de símbolos de origem, permitindo uma análise comparativa entre as duas abordagens.

Chapter 5

Implementação

Chapter 6

Resultados

Chapter 7

Conclusão

Referências

[1] Luca Clissa, Mario Lassnig, Lorenzo Rinaldi: *How big is big data? a comprehensive survey of data production, storage, and streaming in science and industry.* Big Data and AI in High Energy Physics, 6(9), 2023. https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2023.1271639/full. 1, 2

[2] Begen, Ali C., Thomas Stockhammer, and Colin Perkins: *Energy and ai.* Technical report, INTERNATIONAL ENERGY AGENCY, 2025. https://www.iea.org/. 2

[3] Amaral, Dino Macedo: *Hy-SAIL: Uma nova abordagem para distribuição e armazenamento de informações em ambientes de computação em nuvem.* PhD thesis, Universidade de Brasília, 2013. 2

[4] Yu, Shui: *Big privacy: Challenges and opportunities of privacy study in the age of big data.* IEEE Access, 4:2751–2763, 2016. 3

[5] Zhao, Xiaoyan, Conghui Zhang, and Shaopeng Guan: *A data lake-based security transmission and storage scheme for streaming big data.* Cluster Computing, 27(4):4741–4755, Jul 2024, ISSN 1573-7543. https://doi.org/10.1007/s10586-023-04201-9. 3

[6] Hakami, Tahani Ali, Yasser M. Alginahi, and Omar Sabri: *Exploring the evolution of big data technologies: A systematic literature review of trends, challenges, and future directions.* Future Internet, 17(9), 2025, ISSN 1999-5903. https://www.mdpi.com/1999-5903/17/9/427. 3

[7] Villamar-Torres, Ronald Oswaldo, Verónica Cristina Andrade Yucailla, Karen Rafaela Mayorga Morejon, Héctor Adonis Almeida Reyes, and Seyed Mehdi Jazayeri: *An overview to the new era in efficient crop management: Artificial intelligence, machine learning, big data, bioinformatics, metagenomics and precision agriculture.* Journal of Animal and Plant Sciences, 35(3):638–659, June 2025, ISSN 1018-7081. https://thejaps.org.pk/AbstractView.aspx?mid=2024-japs-1928. 3

[8] Ko, Dongyub, Kijong Koo, and Do Young Kim: *Reducing the decoding complexity of raptorq codes for delay sensitive applications using a simplified and scaled-down matrix.* AEU-INTERNATIONAL JOURNAL OF ELECTRONICS AND COMMUNICATIONS, 70(9):1356–1360, 2016, ISSN 1434-8411. 4, 8, 10

[9] Luby, Michael: *Lt codes.* In *Proceedings of the 43rd Symposium on Foundations of Computer Science (FOCS)*, pages 271–280. IEEE Computer Society, 2002. 7

[10] Shokrollahi, Amin: *Raptor codes.* IEEE Transactions on Information Theory, 52(6):2551–2567, 2006. 8, 10

[11] Maymounkov, Petar: *Online codes.* Technical Report TR2002-833, New York University, New York, NY, November 2002. 10, 11

[12] Maymounkov, Petar and David Mazieres: *Rateless codes and big downloads.* In Kaashoek, M. Frans and Ion Stoica (editors): *Peer-to-Peer Systems II*, volume 2735 of *Lecture Notes in Computer Science*, pages 247–255. Springer, 2003. 10